Docker Project – Restaurant Website Deployment (Ubuntu)

Objective

Install Docker on Ubuntu, manage Docker images and containers, work with Docker volumes and networks, deploy a Restaurant Website using Nginx, and verify container operations.

Phase 1 – Prepare Ubuntu Server

Update the system

```
sudo apt update -y

sudo apt upgrade -y

sudo apt install docker.io -y
```

Verify Docker installation

```
docker --version
```

Start and enable Docker

```
sudo systemctl start docker

sudo systemctl enable docker

sudo systemctl status docker
```

Phase 2 – Create Project Directory

```
mkdir restaurant-project

cd restaurant-project
```

Phase 3 – Download Docker Images

Pull Ubuntu image

```
docker pull ubuntu
```

Pull Nginx image

```
docker pull nginx
```

Verify downloaded images

```
docker images
```

Phase 4 – Create Docker Volume

Create a persistent Docker volume

```
docker volume create restaurant-data
```

List available volumes

```
docker volume ls
```

Inspect the volume

```
docker volume inspect restaurant-data
```

Phase 5 – Create Ubuntu Administration Container

Launch Ubuntu container with the shared volume.

```
docker run -it \
--name ubuntu-admin \
-v restaurant-data:/restaurant-data \
ubuntu bash
```

Purpose

- Administration container

- Volume testing
- File management

Exit the container.

```
exit
```

Phase 6 – Deploy Nginx Website Container

```
docker run -d \
--name restaurant-web \
-p 8080:80 \
-v restaurant-data:/usr/share/nginx/html \
nginx
```

Verify

```
docker ps
```

Open shell inside container

```
docker exec -it restaurant-web bash
```

Phase 7 – Monitor Containers

Running containers

```
docker ps
```

All containers

```
docker ps -a
```

View logs

```
docker logs restaurant-web
```

Inspect container

```
docker inspect restaurant-web
```

Restart

```
docker restart restaurant-web
```

Phase 8 – Remove and Recreate Container

Stop

```
docker stop restaurant-web
```

Remove

```
docker rm restaurant-web
```

Create updated container

```
docker run -d \
--name restaurant-web-v2 \
-p 8080:80 \
-v restaurant-data:/usr/share/nginx/html \
nginx
```

Verify

```
docker ps
```

Phase 9 – Configure Docker Without sudo

```
sudo usermod -aG docker $USER
```

Reboot

```
sudo reboot
```

Verify

```
docker ps
```

Phase 10 – Docker Networking

View networks

```
docker network ls
```

Inspect default bridge

```
docker inspect bridge
```

Run container using host network

```
docker run -d \
--name nginx \
--network host \
nginx
```

Phase 11 – Create Custom Docker Network

Create network

```
docker network create app-net
```

Verify

```
docker network ls
```

Deploy first container

```
docker run -d \
--name nginx-cust1 \
--network app-net \
nginx
```

Deploy second container

```
docker run -d \
--name nginx-cus2 \
--network app-net \
nginx
```

Inspect network

```
docker inspect app-net
```

Purpose

- Container-to-container communication
- Custom bridge networking

Phase 12 – Update Restaurant Website

```
cd ~/restaurant-project
```

Create/Edit homepage

```
vi index.html
```

Copy website into container

```
docker cp index.html restaurant-web-v2:/usr/share/nginx/html/index.html
```

Verify files

```
docker exec -it restaurant-web-v2 ls -R /usr/share/nginx/html
```

Phase 13 – Add Images

Create images directory locally

```
mkdir -p images
```

Create images folder inside container

```
docker exec -it restaurant-web-v2 mkdir -p /usr/share/nginx/html/images
```

Copy restaurant image

```
docker cp ~/restaurant-project/images/restaurant.jpg restaurant-web-
v2:/usr/share/nginx/html/images/
```

Verify

```
docker exec -it restaurant-web-v2 bash
```

Phase 14 – Verify Website

Check running container

```
docker ps
```

Find public IP

```
curl ifconfig.me
```

Open in browser:

```
http://<Public-IP>:8080
```

The Restaurant Website should now be accessible.

Phase 15 – Package the Project

Install zip utility

```
sudo apt install zip unzip -y
```

Create archive

```
zip -r restaurant-project.zip restaurant-project
```

Docker Commands Practiced

- Docker Installation
- Docker Images
- Docker Containers
- Docker Volumes
- Docker Networks
- Container Inspection
- Container Logs
- Restarting Containers
- Removing Containers
- Host Networking
- Custom Bridge Networking
- Copying Files into Containers
- Website Deployment with Nginx
- Project Packaging

Skills Demonstrated

- Ubuntu Administration
- Docker Installation & Configuration
- Docker Image Management
- Container Lifecycle Management
- Docker Volumes
- Persistent Storage
- Docker Networking

- Nginx Web Server Deployment
- Static Website Hosting
- Container Troubleshooting
- Linux Command Line
- Website Deployment on Docker